

Using ChatGPT’s Study Mode for Comparing Various Transformer-Based Large Language Models

Report Summary:

In this report, I use ChatGPT’s study mode to compare the strengths and weaknesses of various transformer-based large language models (LLMs), such as GPT, BERT, Llama, SmoLLM3, OLMoE, DeepSeek, Qwen, and Kimi. It classifies these models based on architectural choices and the tasks (e.g., classification, regression, and generation) for which they are most suitable. One feature of the study mode that I found very interesting is that it organizes concepts and aids the user’s understanding by asking questions for each concept. Then, it moves to the next concept and repeats this Socratic tutoring behavior. Finally, it generates a cheat sheet that helps me compare different architectural choices for models.

Model	Encoder/Decoder	Dense vs MoE	Typical strengths (why)
BERT	Encoder-only, bidirectional	Dense	Great for classification / NER / extractive QA → masked LM + bidirectional attention gives strong representations.
GPT-3	Decoder-only, causal	Dense	Fluent long-form generation → trained exactly on next-token prediction.
Llama (e.g. 3.2)	Decoder-only	Dense	General-purpose chat, multilingual, RAG-friendly → modern blocks (RoPE, GLU) tuned for dialogue.
Qwen	Decoder-only	Mostly dense + some MoE variants	Very strong multilingual, code, math → lots of domain-specific training + variants.
SmoLLM3	Decoder-only	Dense, small	On-device / low-resource assistant with surprisingly good reasoning → small but well-trained long-context model.
OLMoE	Decoder-only	MoE	Research/open baseline showing MoE can match larger dense models at similar compute via expert specialization.
DeepSeek (V3/R1)	Decoder-only	MoE	Strong code & math & reasoning → huge MoE + advanced objectives (multi-token prediction, RL for reasoning).
Kimi K2	Decoder-only	MoE	Very large MoE tuned for agentic tasks and coding; big capacity + tool-use/post-training.

Prompt 1:



Lecture 20.pdf
PDF

This is the lecture note of EECS 182 offered at University of California, Berkeley Fall 2025. It discusses about the architecture of GPT-3(decoder-only model), BERT(encoder-only model), Mixture of experts models(OLMO-E), Qwen, Llama, SmolLM3, DeepSeek, and Kimi. I want to compare and contrast among these models. In particular, I want to understand their similarities and differences. For example, the task domains where some models are good at while others are not very good at, and the reasons tracing back to their specific architecture choices.

ChatGPT's answer:

Thought for 2m 50s >

All of these models are really variations on the same basic Transformer, but they sit in *different corners of the design space*. Let me give you a structured "map", then we can zoom in wherever you like.

1. Big taxonomy

Think of three main axes:

1. Encoder vs decoder

- **Encoder-only (BERT)** – sees *all* tokens at once (bidirectional attention), great at *understanding*.
- **Decoder-only (GPT-3, Llama, Qwen, SmolLM3, DeepSeek, Kimi, OLMoE)** – masked/causal attention, great at *generating*.

2. Dense vs Mixture-of-Experts (MoE)

- **Dense** – every token uses *all* parameters in each layer (GPT-3, BERT, Llama, SmolLM3, many Qwen variants).
- **Sparse MoE** – each token is routed to a *few experts* (sub-MLPs) instead of all of them (Switch Transformer, OLMoE, DeepSeek-V3, Qwen2.5-Max, Kimi K2). [Lecture 20](#) [arXiv +4](#)

3. Block choices & positional encoding

- FFN vs **gated MLP / GLU / SwiGLU**, see your slide on gated linear units. [Lecture 20](#)
- Positional encodings: absolute (older GPT/BERT) vs **RoPE/NoPE + YARN** for long context (Llama 3.2, SmolLM3, Qwen2.5, DeepSeek-V3). [Hugging Face +4](#)

2. GPT-3 (decoder-only) vs BERT (encoder-only)

Architecture

GPT-3

- **Decoder-only**, causal self-attention: token t can only attend to $\leq t$. [GitHub](#) +1
- Trained with **next-token prediction** on ~300B tokens; 96 layers, 175B params, context 2048 (your slide). [Lecture 20](#)
- Objective directly matches "generate text left-to-right".

BERT

- **Encoder-only**, fully bidirectional self-attention; tokens attend to both left and right context. [arXiv](#) +2
- Trained with **masked language modeling** (predict ~15% masked tokens) + **next sentence prediction** (NSP). [Lecture 20](#) [arXiv](#) +1
- At fine-tuning time, you attach a small task head (classification, span prediction, etc.) on top of the encoder. [Lecture 20](#)

What they're good at (and why)

- **Natural-language *understanding*** (classification, NER, QA extractive)
 - BERT tends to be simpler & strong: bidirectional attention + [CLS] token naturally give a global representation for the whole sentence or span.
 - GPT-style models *can* do these via prompting, but it's indirect and usually less parameter-efficient for small models.
- **Open-ended *generation*** (chat, story writing, code generation, chain-of-thought)
 - GPT-3 wins: causal LM objective is exactly "predict the next token", so sampling gives fluent text.
 - Vanilla BERT cannot easily generate long coherent sequences (it was never trained as an autoregressive generator).

So: **bidirectional encoder** → **strong discriminative representations**, **causal decoder** → **strong generative behavior**.



3. Dense decoder-only family: GPT-3, Llama, Qwen, SmoLLM3

These are all GPT-style, but with upgraded internals.

Shared traits

- **Decoder-only Transformers** with causal attention.
- Use **RoPE-style positional encodings** (or variants) to support much longer contexts than GPT-3's original 2k. [Hugging Face +2](#)
- Use **gated MLPs (SwiGLU/SiLU-GLU)** instead of vanilla FFNs for better expressivity per parameter (your GLU slide + modern architectures). [Lecture 20](#) [LearnOpenCV +1](#)

Main "personalities"

- **Llama 3.2 (1B / 3B)** – dense, multilingual chat models optimized for dialogue, retrieval-augmented generation, and summarization. Uses RoPE, RMSNorm, SwiGLU, and grouped-query attention; tuned for small-to-medium deployment and strong multilingual chat performance. [Hugging Face +2](#)
- **Qwen (esp. Qwen2.5 dense models)** – similar decoder stack, but strong focus on **Chinese + English + many languages**, long context, and coding/math benchmarks; many specialized variants (coder, math, audio, vision). [arXiv +1](#)
- **SmoLLM3 (3B)** – dense, 3B-param, open model explicitly targeting **on-device use, long context (128k), and reasoning**. Uses NoPE/RoPE+YARN tricks for long context and extra mid-training on reasoning data. [Hugging Face +3](#)

Task tendencies

- All of these are **general-purpose chat / generation** models.
- **Smaller dense models** (Llama 3.2 1B/3B, SmoLLM3) are good when:
 - Latency / memory is tight (mobile, local GPU).
 - You still want decent reasoning and multilingual ability.
- **Qwen dense models** shine on:
 - Multilingual tasks (esp. Chinese).
 - Coding and math (many specialized variants,  rich training data). [Novita +1](#)

4. Mixture-of-Experts: Switch, OLMoE, DeepSeek-V3, Qwen2.5-Max, Kimi K2

Your notes show the **Switch Transformer** and **OLMoE** diagrams: the dense FFN is replaced by many expert FFNs plus a router that picks a few per token. Lecture 20

Key idea:

Total parameters ↑, active parameters per token ≈ constant
→ cheat scaling laws: more capacity with roughly the same FLOPs.

OLMoE (OLMo-E)

- Open 7B-total-parameter MoE with ~1B active per token; experts only in FFN blocks. arXiv +2
- Competes with larger dense models (e.g., Llama-2-13B) at similar *compute* cost.

Good at: getting “big-model” quality for lower inference cost; nice lab model for studying expert specialization.

DeepSeek-V3

- 671B total parameters, ~37B active, MoE + Multi-head Latent Attention (MLA) and a multi-token prediction objective. arXiv +2
- Training objective predicts several future tokens at once → better training efficiency and often stronger reasoning at same compute.

Good at:

- General chat + strong coding and math.
- Efficient large-scale deployment (MoE + MLA).
- Underlies the **DeepSeek-R1** reasoning models, which add heavy RL for chain-of-thought. arXiv +2

Qwen2.5-Max / Turbo (MoE)

- Large Qwen2.5 variants that use MoE layers instead of pure dense MLPs; parameter counts ~325B with sparse activation. arXiv +2

Good at:

- High-end reasoning, coding, multilingual chat with better cost/performance vs equally strong dense models.

Kimi K2

- ~1T parameters, ~32B active, MoE Transformer very similar structurally to DeepSeek-V3 but with **more experts and fewer MLA heads**. Kimi K2 +2
- Uses a new **Muon/MuonClip optimizer** to stabilize training and efficiently reach this scale. Kimi K2 +2
- Post-training heavily emphasizes **agentic tool use** – synthetic data where the model calls tools, plans, and executes tasks. Baseten +1

Good at:

- Complex **agentic workflows** (planning, tool use, multi-step tasks).
- Top-tier coding performance among open models. Substack

Why MoE changes behavior

- **Specialization:** different experts can focus on code, math, multilingual text, etc. You often see higher performance on specialized benchmarks for the same *active* parameter count. [arXiv +3](#)
- **Trade-offs:**
 - Harder to train (routing stability, load balancing).
 - Failure modes like “wrong expert routed → weird answer”.
 - Inference infra more complex (expert sharding, communication).

Dense models don't get this specialization, but they're simpler and sometimes more robust on out-of-distribution inputs.

5. Qwen vs Llama vs SmolLM3 vs DeepSeek vs Kimi – what's different in practice?

On your last slide (Sebastian Raschka's comparison), each model's block diagram tweaks the same template: embeddings → [Norm → Attention → Norm → MLP]×N. [Lecture 20](#)

High-level tendencies:

- **Llama 3.2**
 - “Clean” dense decoder stack, Swish-GLU MLPs, RoPE, RMSNorm.
 - Tuned heavily for *multilingual dialogue* and RAG tasks. [Hugging Face +2](#)
- **Qwen2.5**
 - Similar core, but often mixes in **gated attention / MoE MLPs** and long-context tricks; stronger focus on **coding, math, and multilingual** benchmarks. [arXiv +2](#)
- **SmolLM3**
 - Small dense core, optimized for **128k context** and **reasoning-oriented training**, meant to run on edge devices with good performance. [Hugging Face +2](#)
- **DeepSeek-V3 / R1**
 - Big MoE + MLA; **multi-token prediction + RL reasoning** for R1 → strong math, coding, and chain-of-thought. [arXiv +3](#)
- **Kimi K2**
 - Similar MoE+MLA family but even larger (1T), with optimizer + post-training tuned for **agentic tasks** and coding. [Kimi K2 +3](#)

6. Putting it all together (very compressed)

You can roughly place them in this grid:

Type	Examples	Best for	Architectural reason
Encoder-only dense	BERT	Classification, NER, extractive QA	Bidirectional attention + masked LM → rich token representations; simple heads.
Decoder-only dense (small/medium)	GPT-3 (historical), Llama 3.2 1B/3B, SmoLM3 3B, dense Qwen	General chat, summarization, code, long-context (modern ones) when you need simple infra	Causal LM objective + upgraded GLU/positional encodings; all params active → simplicity.
Decoder-only MoE	OLMoE, DeepSeek-V3, Qwen2.5-Max, Kimi K2	High-end reasoning/coding/chat at lower per-token FLOPs	Sparse experts give much more capacity per unit compute; experts specialize.
Decoder-only + RL reasoning	DeepSeek-R1, some SmoLM3 / Qwen "reasoning" modes	Hard math, logic, code; chain-of-thought	RL rewards correct multi-step reasoning, shaping how attention + MLPs are used.

Quick check-in

The *most important* conceptual contrast is:

BERT-style (encoder, bidirectional, masked LM) vs GPT-style (decoder, causal, next-token).

Everything else (MoE vs dense, GLU vs FFN, long-context tricks) is "fine tuning" of this base.

Could you briefly explain, in your own words, **why BERT is naturally better for classification, while GPT-3-style models are naturally better for generation?** Once I see your answer, we can zoom into any pair you want (e.g., DeepSeek-V3 vs Kimi K2, or Llama vs Qwen) at a deeper architectural level.

Comment 1: ChatGPT can classify the models listed in the lecture notes based on their architectural choices. Then, it provides detailed information for each large language model, with the specific tasks it is best suited for. It then follows up with a question asking me to explain why BERT is naturally better for classification, while GPT-3-style models are naturally better for generation tasks.

Prompt 2.1:

I think it's because BERT is an encoder-only model that uses non-causal self-attention to learn the important features of the training data. So, the learned features serve as a great tool for classification tasks. However, the GPT-3 style model is a decoder-only model that uses causal self-attention to maximize the probability of next token prediction. So, it's desired for generation tasks.

ChatGPT's answer:

Yes, that's exactly the core idea 🧠*

Let me just sharpen your wording a tiny bit:

- **BERT (encoder-only, bidirectional attention):**

Trained to *reconstruct* masked tokens using **both left and right context**. That forces it to build rich contextual features for every token and for the whole sequence (e.g., the [CLS] representation). Those features are perfect for **classification / span prediction** once you slap a small head on top.

- **GPT-3-style (decoder-only, causal attention):**

Trained to **predict the next token** given *only past tokens*. That objective is exactly what we want at test time when we generate text token by token → fluent **generation** is "baked into" the training.

So your explanation is on point 👍

Now, to move toward your bigger goal (comparing GPT-3, BERT, OLMo-E, Qwen, Llama, SmoLLM3, DeepSeek, Kimi), the next big axis after encoder vs decoder is:

Dense vs Mixture-of-Experts (MoE)

Here's a quick intuitive picture:

- **Dense layer:** every token always uses **the same MLP** in that layer.
- **MoE layer:** there are **many expert MLPs**, and a small router chooses (say) the top-2 experts per token. So:
 - total parameters are huge (lots of experts),
 - but **only a few experts are active per token**, so per-token FLOPs stay similar.

This is what distinguishes things like **OLMo-E / DeepSeek / Kimi (MoE)** from dense models like **GPT-3 / Llama / SmoLLM3 / some Qwen variants**.

Quick question for you (next step):

If each token in an MoE model only uses a *subset* of the experts, what does that buy you compared to a dense model with the same per-token compute?

In other words:

What advantage do you get by having many experts but routing each token to only a few of them?

Give me your intuition, and then we'll tie that to why MoE models (DeepSeek, Kimi, OLMo-E) can be so strong on certain tasks (like coding / math / "agentic" behavior) while staying relatively compute-efficient.

Comment 2.1: ChatGPT explains different model architectural choices one by one, asking me a question about each.

Prompt 2.2:

It will greatly save computational cost while generating the tokens.

ChatGPT's answer:

Thought for 18s >

Yep, that's one of the big wins ✓

If we compare to a dense model with the **same per-token FLOPs**, MoE buys you **two things at once**:

1. Same (or slightly more) compute per token

- Each token only passes through, say, **2 out of 64 experts** in that layer.
- So the cost is like using **2 MLPs**, not 64 → compute stays manageable.

2. Huge *total* capacity + specialization

- Even though any *one* token only sees a few experts, the **whole model** has way more parameters.
- The router learns patterns like:
 - "These tokens look like **Python code** → send to 'code' experts."
 - "These look like **Chinese text** → send to multilingual experts."
 - "These look like **math / reasoning** → send to more 'logical' experts."
- So different experts can **specialize**, instead of one big MLP having to do everything.

That's why models like **DeepSeek / Kimi / OLMoE** can:

- Match or beat much larger dense models on **coding, math, multi-step reasoning**,
- While still being relatively efficient at inference.

Let me bounce this back to you:

If some experts specialize in "code-like" tokens and others in "natural language" tokens, how might that affect performance on **pure coding tasks** compared to a same-FLOPs dense model with no experts?

Try to answer in 1–2 sentences, just your intuition. Then we'll zoom out and connect this to concrete differences between, say, **Llama / SmoLM3 (dense)** and **DeepSeek / Kimi (MoE)** in terms of which tasks they tend to excel at.

Comment 2.2: ChatGPT constructs a concrete scenario to help me understand the differences between sparse mixture-of-experts models and dense MLPs.

Prompt 3.1:

It might perform worse compared to dense model due to fewer number of parameters to tune.

ChatGPT's answer:

Thought for 30s >

Nice, this is a good time to fix a subtle but important intuition 😊

Where your reasoning is slightly off

In a typical **MoE layer** (like in OLMoE / DeepSeek / Kimi):

- There are, say, **64 experts** (64 different MLPs).
- For each token, the router picks **top-k experts** (e.g., $k = 2$).
- So **only 2 experts run per token**.

Now compare that to a **dense** model with the *same FLOPs per token*:

- **Active parameters per token** $\approx$ the same (1–2 “experts’ worth” of MLP).
- But the MoE model’s **total parameters** are *much larger* because it has many experts, even though each token only touches a few.

So:

You **don’t** have fewer parameters to tune per token *compared to a same-compute dense model*.
You have **similar active parameters** but **much larger total capacity**, which lets experts *specialize*.

For pure coding tasks, this usually helps:

- Some experts specialize on **code-like tokens** (Python, C++, etc.).
- Others specialize on **natural language**.
- When the input is pure code, the router tends to send many tokens to the “code-ish” experts.
→ You effectively get a “code-specialist sub-network” with more relevant parameters than a single shared dense MLP.

That’s one reason MoE models like **DeepSeek / Kimi / big Qwen MoE** are often very strong on **code** at similar or lower per-token compute than an equivalently strong dense model.

Quick check-in question

To connect this to your original big picture (GPT-3, BERT, Llama, Qwen, SmoLLM3, DeepSeek, Kimi):

Imagine you're designing a system and must pick **one** model for each scenario:

1. **On-device assistant on a phone** (tight memory + compute).
2. **Cloud coding assistant** for heavy-duty software engineering work.

Which *type* would you lean toward in each case, and why?

- (a) Small **dense** decoder-only (e.g., SmoLLM3 / tiny Llama).
- (b) Large **MoE** decoder-only (e.g., DeepSeek / Kimi / big Qwen MoE).

Just pick (a) or (b) for each scenario and give a 1–2 sentence justification. Then we can map each of the named models (GPT-3, BERT, OLMoE, Llama, Qwen, SmoLLM3, DeepSeek, Kimi) into those use-cases and tie it more concretely to their architectures.

Comment 3.1: When I made a mistake in answering ChatGPT's question, it pointed out where I made my mistake and explained the correct reasoning steps. Then it asks me a follow-up question that deepens my understanding.

Prompt 3.2:

1. pick b) due to memory + compute constraint and the possibility of receiving many different types of tasks. 2. Pick a) due to heavy-duty and flexibility on memory/compute constraints.

ChatGPT's answer:

Thought for 14s >

Nice, thanks for committing to an answer — but this is one of those cases where the *intuition flips* a bit 😊

You've basically **swapped** the choices:

1. On-device assistant (phone, very tight memory/compute)

Here you actually want: **(a) small dense decoder-only** (SmolLM3 / tiny Llama).

Why *not* MoE here?

- Even though MoE saves **compute per token**, you still have to **store all the experts' weights** on the device.
- MoE models have **huge total parameters** (e.g., 300B–1T), just sparsely activated.
- A phone can't realistically hold that in RAM/VRAM, even if only 1–2 experts are used per token.

So for a phone:

Dense small model = fewer total parameters → fits in memory, simpler inference stack.

2. Cloud coding assistant (heavy-duty, server-side)

Here you want: **(b) large MoE decoder-only** (DeepSeek, Kimi, big Qwen MoE).

Why MoE now?

- In a data center, you can **shard experts across many GPUs**.
- You care about **quality per FLOP** and **throughput**, not fitting on a single tiny device.
- MoE lets you have **very high total capacity** (experts specializing in code, math, etc.) while keeping per-token compute manageable.

So for cloud:

MoE big model = massive capacity + specialization → excellent coding / reasoning at reasonable compute cost.

So the rule-of-thumb:

- **Edge / on-device** → small **dense** model.
 - **Cloud / high-end service** → large **MoE** model (or very big dense if you're willing to pay more compute).
-

Quick check for you

Can you summarize in **one sentence** why MoE is *bad* for phones but *great* for cloud servers?

Comment 3.2: It creates a concrete example that helped me to understand the pros and cons of MoE models. To conclude the model architectural choices comparison, I am asked to summarize in one sentence.

Prompt 3.3:

Phones typically have restricted memory/compute resources and less demand for using many different experts, however, cloud servers have flexible memory/compute resources and more demand for using many different experts.

ChatGPT's answer:

Thought for 20s >

Nice, that's very close 🤝 Let me tweak it just a bit:

- The **main reason MoE is bad for phones** isn't really "less demand for many experts," it's:
 - You still have to **store all the experts' weights** → *huge total memory footprint*.
 - Phones have tiny RAM/VRAM and simple runtimes, so MoE's routing + sharding complexity is a bad fit.
- **Cloud servers**, on the other hand:
 - Can **shard experts over many GPUs** and keep them in memory.
 - Serve **lots of users and diverse tasks**, so the extra specialization from many experts actually pays off.

A tighter version of your sentence could be:

MoE is bad for phones because it has huge total parameter memory and complex routing, but great for cloud servers where we can shard that big MoE across many GPUs and fully exploit specialized experts for diverse workloads.

Model	Encoder/Decoder	Dense vs MoE	Typical strengths (why)
BERT	Encoder-only, bidirectional	Dense	Great for classification / NER / extractive QA → masked LM + bidirectional attention gives strong representations.
GPT-3	Decoder-only, causal	Dense	Fluent long-form generation → trained exactly on next-token prediction.
Llama (e.g. 3.2)	Decoder-only	Dense	General-purpose chat, multilingual, RAG-friendly → modern blocks (RoPE, GLU) tuned for dialogue.
Qwen	Decoder-only	Mostly dense + some MoE variants	Very strong multilingual, code, math → lots of domain-specific training + variants.
SmolLM3	Decoder-only	Dense, small	On-device / low-resource assistant with surprisingly good reasoning → small but well-trained long-context model.
OLMoE	Decoder-only	MoE	Research/open baseline showing MoE can match larger dense models at similar compute via expert specialization.
DeepSeek (V3/R1)	Decoder-only	MoE	Strong code & math & reasoning → huge MoE + advanced objectives (multi-token prediction, RL for reasoning).
Kimi K2	Decoder-only	MoE	Very large MoE tuned for agentic tasks and coding; big capacity + tool-use/post-training.

To make sure this really clicked:

If you wanted to build a **pure text classifier** (say, sentiment or topic) and you're allowed to *fine-tune*, which family would you start from: **BERT-style encoder** or **GPT-style decoder**, and why?

One or two sentences is enough.



Comment 3.3: It constructs a table listing all transformer-based models and comparing their architectural choices and strengths for the tasks they are suited for.

Prompt 3.4:

Since the goal is to classify, I would choose a BERT-style encoder

ChatGPT's answer:

Thought for 39s >

Exactly ✓

For a pure classifier, a **BERT-style encoder** is the natural starting point because:

- It builds a **single, bidirectional representation** of the whole input (e.g., the **[CLS]** vector).
- It's trained to **understand** the sequence (masked LM), not to continue it.
- You can just stick a tiny linear layer on top and fine-tune → very sample-efficient for classification.

A GPT-style decoder *can* do classification via prompting ("Answer: positive/negative"), but it's more indirect and usually less parameter-efficient than fine-tuning a BERT encoder for that job.

Now that we've nailed:

1. **Encoder vs decoder** (BERT vs GPT-3 family), and
2. **Dense vs MoE** (Llama/SmolLM3 vs DeepSeek/Kimi/OLMoE),

let's start organizing **all 8 models** on those two axes so we can really compare them.

Here are the axes again:

- Axis 1: **Encoder-only** vs **Decoder-only**
- Axis 2: **Dense** vs **MoE**

🕒 Your turn (small exercise):

For each of these models, can you fill in *just* the 2 labels:

(encoder/decoder, dense/MoE)

- BERT
- GPT-3
- Llama
- Qwen
- SmolLM3
- OLMoE
- DeepSeek
- Kimi

You can answer in a quick list like:

"BERT: encoder, dense; GPT-3: decoder, dense; ..."

Then I'll correct anything off and we'll move to:

"Given those categories, *which* tasks each family tends to be best at, and *why* (architecture → behavior).

Comment 3.4: Finally, it asks me to classify the models listed in the lecture notes by their architectural choices.